

AI System Maintenance and Update Strategies

ARTI409 - SAIT Summer Semester 2026

Authors: Roy Aggarwal, Gurpreet Bajwa, Chuck Burns, Neil McGuffin, Sukhpreet Singh
Instructor: Tim Cruz

Introduction

Production AI systems are not static.. Their performance can deteriorate as data distributions, user behaviour, regulations, infrastructure, and business objectives change. The literature commonly uses the term continual learning for models that incrementally acquire knowledge after deployment. In this report, continuous learning is used in that same broad sense. Its central challenge is preserving previously learned capabilities while remaining adaptable to new information, often described as the stability–plasticity trade-off [1]. Concept drift presents a related maintenance problem because changes in the relationship between model inputs and outcomes can gradually reduce the relevance of an apparently healthy model unless those changes are detected, evaluated, and addressed [2].

This report examines two connected problems. First, it critically compares continuous learning and scheduled updating as AI maintenance paradigms, assessing their efficiency, effectiveness, risks, and mitigation controls. Second, it develops a risk-based maintenance and update strategy for a hypothetical autonomous-vehicle fleet operating across jurisdictions in the United States. The strategy addresses how the fleet can incorporate new traffic regulations while protecting safety, maintaining regulatory compliance, and minimizing disruption to vehicle availability. Although the report does not apply a single formal risk-management framework, its recommendations emphasize structured governance, testing, monitoring, documentation, and proportionate controls.

The report concludes that neither continuous learning nor scheduled updating is universally superior. Continuous learning can improve responsiveness to emerging conditions, but it introduces stability, governance, and validation challenges. Scheduled updating offers stronger control, reproducibility, and auditability, but it can allow deployed models to become stale between

release cycles. For safety-critical systems, a controlled hybrid is therefore generally the strongest approach with continuous monitoring and data collection, trigger-based development of candidate updates, rigorous offline and shadow evaluation, staged deployment, and defined oversight. The autonomous-vehicle strategy follows this principle by treating updates as controlled safety-relevant changes that require appropriate evidence, review, monitoring, and rollback capability rather than as ordinary software patches.

Analytical Comparison of AI Update Methodologies

Before the two paradigms can be compared meaningfully, the terms need to be fixed. Continuous learning updates the model incrementally as new data arrives, so the weights are never static for long. Scheduled updates hold changes back, train and validate a full new version offline, and release it at a fixed interval. Neither paradigm is categorically better; each is a different answer to the same underlying question, namely how much a system can be allowed to change before that change itself becomes the risk.

Efficiency

Continuous learning integrates new information almost immediately, so the model is rarely behind the data it is meant to reflect. This responsiveness is not free. A persistent training pipeline has to run alongside inference at all times, and oversight becomes an ongoing task rather than a scheduled one, since a problem can surface at any point rather than at a known checkpoint. Scheduled updates invert this trade-off. Because retraining and validation happen in a single planned cycle, compute and engineering review can be budgeted in advance, and a team is evaluating one coherent release rather than a continuous stream of small changes. The cost of that predictability is staleness between cycles: the model does not know what it does not yet know, and will not know it until the next release.

Effectiveness

Where the operating environment drifts gradually, continuous learning tends to hold its performance, since the model keeps adjusting rather than falling behind the data. Its central weakness is catastrophic forgetting: as the network absorbs new patterns, it can lose previously learned ones, because extending a trained network's knowledge without targeting the original task tends to degrade performance on that task [1]. Scheduled updates avoid this particular instability, since each release is validated as a complete, self-contained version before deployment rather than left to shift continuously. Their failure mode sits on the other side of the same coin: if the environment changes faster than the release cycle, the deployed model quietly falls out of step with it. This is generally described as concept drift, where the relationship between the input data and the target the model is predicting shifts over time, and a model left unrevised keeps making decisions calibrated to a world that no longer exists [2]. A more recent strand of research sits between the two paradigms entirely: rather than treating the release schedule as fixed, drift-aware systems monitor live performance against a baseline and use a detected drift event itself as the trigger to pull a retraining cycle forward, effectively making the schedule adaptive rather than calendar-based. This does not resolve the underlying trade-off, but it narrows the gap between the two paradigms rather than forcing a strict choice between them.

Risks and Mitigations

Continuous learning carries two risks worth naming directly: bias creep and forgetting. A model trained continuously on live data can absorb noise, outliers, or a shift in the population it is exposed to, and none of it appears as a single identifiable bad update, it accumulates as a slow skew rather than a clean failure. This is mitigated with replay buffers that periodically re-expose the model to earlier data, regularization that constrains how far existing weights are allowed to

move, and staged rollouts that limit how much traffic a new update touches before it is fully trusted. Scheduled updates carry the opposite risk. Because changes are batched, a flawed release affects the entire system at once rather than a limited slice, and the flaw persists until the next window opens. This is mitigated with shadow-mode testing and canary releases ahead of full deployment, and by shortening the update interval specifically for the components most exposed to change.

Hybrid Approaches

In practice, systems that need to remain both current and stable tend to combine the two paradigms rather than choose between them: a scheduled, thoroughly validated backbone for the core model, paired with a lighter continuous-learning layer for whatever changes fastest, such as recent behavioural signals or an emerging fraud pattern. The backbone supplies a stable, auditable foundation and a defined review point; the fast layer prevents the system from growing stale in the interval between releases.

Consider a fraud-detection system as a concrete case. A scheduled-only design retrains monthly. Partway through a cycle, a new laundering pattern emerges that does not resemble anything in the training data; for the remaining two or three weeks until the next release, the model has no mechanism to catch it, and every case that pattern touches goes undetected in the interim, a direct, quantifiable cost of the staleness described above. A continuous-learning layer closes that specific gap, since it can begin adjusting to the new pattern within hours of first seeing it rather than waiting for the calendar. Now take the opposite case: a medical diagnosis model operating under a fixed regulatory approval. Here, continuous learning is a liability rather than an asset, since a model that drifts even slightly between approvals is no longer the model that was validated and cleared, and any resulting misdiagnosis is far more costly than a fraud pattern going briefly undetected. A scheduled cycle, where each version is formally revalidated before release, is the

only approach that keeps the system both safe and auditable. The same underlying trade-off, applied to two different domains, produces two opposite answers.

This is also where future-proofing and update strategy converge: a hybrid system does not just balance responsiveness against stability at a fixed point in time, it retains the ability to shift that balance as the domain's own risk profile changes, adding a faster layer if a compliance-critical system starts operating in a more volatile environment, or tightening review if a high-velocity system moves into a more regulated one. A hybrid design earns its added complexity specifically where a system faces both pressures at once, or is likely to migrate between them, which is increasingly the default case for deployed AI rather than the exception.

Strategic Planning for AI Updates in Autonomous Vehicles

Before strategic planning can be discussed meaningfully, the tactical autonomous vehicle (AV) fleet landscape needs to be understood. This portion of the paper studies the autonomous vehicles environment in the United States, since autonomous vehicle adoption and population base in the US is far more widespread than in Canada. In the United States, an autonomous vehicle (AV) fleet answers to two regulators at once, which shapes how updates should be managed. NHTSA sets Federal Motor Vehicle Safety Standards (FMVSS) governing the vehicle itself, while individual states control the rules of the road, licensing, and AV testing permits [3]. A “new traffic regulation” is therefore rarely nationwide; it is usually one state changing a right-of-way rule or a city adjusting an ordinance. A fleet crossing state lines needs an update strategy built around that patchwork, not one uniform rulebook. This plan treats downtime, compliance, testing, communication, and future-proofing as one connected system, names a specific risk and mitigation for each, and accepts a trade-off up front: the rigor built into every stage costs time and resources a cost-optimized rollout would skip. For safety-critical AVs under dual federal and state oversight, that cost is the price of legitimacy, not overhead.

Minimizing Downtime

New AI models are pushed over the air (OTA) and run in shadow mode: the software logs decisions in the background while the certified system keeps physical control. Since state law is what usually changes, updates can be geofenced, applied only to vehicles in the affected state, rather than pushed fleet-wide. Once shadow performance matches or beats the live system, the swap happens during overnight depot charging, preserving daytime availability. Two risks apply. Shadow mode only observes; it cannot reveal control-loop failures, such as actuator delay, that appear only once a model actually drives, so closed-track testing is required before any model is trusted with real control. And OTA delivery is a known attack surface: a spoofed update pushed to a fleet is a cybersecurity risk [4], mitigated with cryptographically signed updates verified on-device, so an unauthorized package is rejected automatically.

Ensuring Compliance and Safety

A learning-based model is a statistical system, a risky place to hard-code a legal rule like a new speed limit. This plan separates perception, handled by the AI, from a deterministic, jurisdiction-aware rule layer that checks every proposed action against the law of the state the vehicle is currently in, in the spirit of the safety envelope defined in IEEE Standard 2846 [5]. When a state changes its law, engineers update that state's entry directly, rather than retraining the model. This also supports NHTSA's Standing General Order, which requires reporting certain automation-related crashes within a defined window [6]; since the rule layer logs every override, the data needed for that federal obligation is captured as a byproduct of normal operation. The risk is disagreement between layers in an untested edge case; the rule layer always holds override authority, and every override is logged and fed back into the simulation suite so the scenario is covered next cycle.

Testing and Validation

Updates pass through simulation, closed-track testing, live shadow mode, and a staged rollout starting near five percent of affected vehicles, with automatic rollback available throughout. No single stage is sufficient alone, so a vehicle only advances once it clears defined thresholds at the prior one. A related risk emerges after deployment rather than during it: model drift, where performance quietly degrades as real-world conditions shift away from the data a model was trained on [7]. This is mitigated with continuous post-deployment monitoring against defined performance baselines, triggering retraining before drift becomes a safety issue rather than after. One forward-looking addition: any vehicle that encounters a novel edge case, an unfamiliar sign or an ambiguous right-of-way conflict, flags it to a shared fleet-wide case library in near real time, tagged to the state or municipality it came from, so every vehicle's simulation suite is updated before the next validation pass.

Stakeholder Communication

Two regulatory audiences need distinct treatment. NHTSA receives federal compliance evidence tied to FMVSS and crash-reporting obligations, reflecting the interdisciplinary approach Koopman and Wagner argue AV safety requires [8]. State DOTs and motor vehicle agencies receive jurisdiction-specific evidence that a vehicle's rule layer correctly reflects their laws. Passengers get a simple in-vehicle notice explaining unfamiliar manoeuvres, and the public receives an annual plain-language safety report. Communication has so far been one-directional; this plan adds a standing feedback channel, an in-app option for passengers and a direct regulator query line, so concerns surface before they escalate. The related risk is silence after a rollback; this is mitigated by a pre-agreed disclosure timeline to both federal and state regulators, so no one hears about a problem from news coverage first.

Future-Proofing

Splitting the AI into a learning-based perception layer and an editable, per-jurisdiction rules layer means most legal changes are handled by updating one state's entry, without retraining models or touching other jurisdictions. Automated monitoring tracks legislative activity across all fifty states plus federal rulemaking, giving engineering teams months of lead time. The residual risk is that fifty-plus jurisdictions are too many for automation to catch everything, particularly informal local changes; this is mitigated with a standing legal-liaison network in each state where the fleet operates, confirming changes through a human channel rather than automation alone.

Conclusion

No single AI maintenance strategy is universally optimal. Instead, organizations must continually balance the trade-offs between continuous and scheduled learning according to their operational, regulatory, and risk requirements. The autonomous vehicle case study demonstrates how these trade-offs manifest in a safety-critical domain, where technical performance must be considered alongside compliance, cybersecurity, and public safety. As AI systems become increasingly integrated into regulated industries, maintaining them has become an interdisciplinary engineering effort involving machine learning, cybersecurity, software engineering, legal compliance, and human oversight. Looking ahead, effective AI maintenance strategies will increasingly rely on adaptive monitoring, modular system architectures, and hybrid update methodologies that balance responsiveness with safety, transparency, and regulatory compliance.

References

- [1] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars, "A continual learning survey: Defying forgetting in classification tasks," IEEE

Transactions on Pattern Analysis and Machine Intelligence, vol. 44, no. 7, pp. 3366–3385, Jul. 2022.

[2] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," IEEE Transactions on Knowledge and Data Engineering, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.

[3] National Highway Traffic Safety Administration, Automated Driving Systems 2.0: A Vision for Safety, Report No. DOT HS 812 442, U.S. Department of Transportation, Washington, DC, Sept. 2017.

[4] S. Halder, A. Ghosal, and M. Conti, "Secure over-the-air software updates in connected vehicles: A survey," Computer Networks, vol. 178, p. 107343, 2020.

[5] IEEE Standard for Assumptions in Safety-Related Models for Automated Driving Systems, IEEE Std 2846-2022, pp. 1–59, 2022.

[6] National Highway Traffic Safety Administration, Standing General Order 2021-01: Incident Reporting for Automated Driving Systems and Level 2 Advanced Driver Assistance Systems, U.S. Department of Transportation, Washington, DC, 2021 (as amended 2025).

[7] F. Bayram, B. S. Ahmed, and A. Kassler, "From concept drift to model degradation: An overview on performance-aware drift detectors," Knowledge-Based Systems, vol. 245, p. 108632, 2022.

[8] P. Koopman and M. Wagner, "Autonomous vehicle safety: An interdisciplinary challenge," IEEE Intelligent Transportation Systems Magazine, vol. 9, no. 1, pp. 90–96, Spring 2017.

AI Usage Statement

Generative AI was used to assist with formatting, editing, and research on this paper. All analysis, source verification, and final content were reviewed and approved by the authors.